Problem Set 7: Classes and Recursion

COSC 120 – Prof. Jordan C. Hanson

Goal: Review the class concepts from Chapter 11 and the recursive patterns from Chapter 12 using problems closely related to our warm-up and in-class exercises.
Submit: Your Python files and a PDF containing the requested written answers, traces, and explanations.

1. A 2D Particle Class

Create a module named

```
particle2d.py
```

containing a class

```
class Particle2D:
```

with the four instance attributes

$$x, \qquad y, \qquad v_x, \qquad v_y.$$

Your constructor should have the form

```
def __init__(self, x, y, vx, vy):
    ...
```

Add the following methods.

A. Update the particle state

```
def update(self, dt):
    ...
```

Use

$$x_{\text{new}} = x + v_x \Delta t, \qquad y_{\text{new}} = y + v_y \Delta t.$$

The method should change the object's position but leave its velocity unchanged.

B. Return the particle speed

```
def speed(self):
    ...
```

using

$$v = \sqrt{v_x^2 + v_y^2}.$$

C. Return the distance from the origin

Add

```
def distance(self):
    ...
```

using

$$r = \sqrt{x^2 + y^2}.$$

Create a second file,

```
test_particle.py
```

that imports the class and creates

```
p1 = Particle2D(1, 2, 3, -1)
p2 = Particle2D(-2, 0, 1, 2)
```

For each particle:

1. print its initial position, speed, and distance from the origin;
2. call `update(0.5)`;
3. print its new position and distance.

Explain briefly why updating `p1` does not change `p2`.

2. Recursion with a List

Write a recursive function

```
def all_nonnegative(values):
    ...
```

that returns `True` only if every number in the list is greater than or equal to zero.
Your function should follow the same recursive pattern used in class:

- the **base case** occurs when no values remain;
- examine one element using `pop()`;
- if that element fails the test, stop immediately;
- otherwise call the same function on the smaller list.

Test the function using

```
a = [4, 9, 0, 12, 3, 7]
b = [4, 9, -2, 12, 3, 7]
```

Before running your code, predict the result for each list.
Then answer:

1. What is the base case?
2. How does each recursive call make the problem smaller?
3. Why can the function stop early when a negative number is found?

3. Recursive Factorial

Write

```
def factorial(n):
    ...
```

using recursion.
Your function should use

$$0! = 1$$

as the base case and

$$n! = n(n-1)!$$

as the recursive case.

A. Compute and print

```
factorial(7)
```

B. Show the recursive expansion of 5!, beginning with

$$5! = 5(4!)$$

and continuing until the base case is reached.

C. Add a conditional so that a negative input such as

```
factorial(-3)
```

does not recurse forever. Your function may return `None` for a negative input.
Explain why a recursive function needs a base case.

4. Recursive Binary Search

Use the sorted list

```
values = [2, 6, 9, 13, 18,
          21, 27, 31, 36, 42,
          48, 55, 63, 72, 84]
```

Write a recursive binary-search function

```
def binary_search(values, target,
                  low, high):
    ...
```

that keeps the original list and changes only the indices `low` and `high`.

Use

$$\texttt{mid} = \left\lfloor \frac{\texttt{low} + \texttt{high}}{2} \right\rfloor.$$

Your function must:

- return `-1` when `low > high`;
- return `mid` when the target is found;
- recursively search the left half when the target is smaller;
- recursively search the right half when the target is larger.

A. Search for a value that is present.

Call

```
binary_search(
    values, 63, 0, len(values)-1)
```

and record the index returned.

Complete a trace with columns

`low`, `high`, `mid`, `values[mid]`, `keep`.

In the `keep` column, write **left**, **right**, or **found**.

B. Search for a missing value.

Call

```
binary_search(
    values, 50, 0, len(values)-1)
```

and record the sequence of midpoint values examined.

What should the function return?

C. Explain the algorithm.

In two or three sentences, explain why binary search requires the list to be sorted and why it is more efficient than checking every element one at a time for a large sorted list.

5. Files to Submit

Submit:

```
particle2d.py
test_particle.py
recursion_tools.py
problem_set_7.pdf
```

Place `all_nonnegative()`, `factorial()`, and `binary_search()` in `recursion_tools.py`.

Your PDF should contain the requested predictions, recursive traces, and short explanations.